MODERN IMAGE PROCESSING

CHAPTER 4 • PART 2 OF 2

Multiresolution Image Representations

From Pyramids to Haar → Looking Ahead

Lecture Notes

In This Part

01 From Pyramids to Haar

02 The Haar Transform

03 Sparsity and Image Compression

04 Beyond Haar

05 Image Compression

06 Looking Ahead

SECTION 8

From Pyramids to Haar

From Pyramids to Haar

- Strip the two pyramids to their essences.
- The Gaussian pyramid is built from one operation: averaging (Gaussian smoothing is weighted local averaging, §3.5).
- The Laplacian pyramid is built from one operation on top of it: differencing (each level is a difference between the image and its average-based prediction).
- Averages carry the coarse content; differences carry the detail.
- Every construction of this chapter has been some arrangement of these two.

From Pyramids to Haar

- The arrangement, however, has been sequential — average first, difference afterward, with the awkward expand shuttling between resolutions and the 33% redundancy as the price.
- So ask the question this section exists for:

Can averaging and differencing be performed simultaneously — as one operation, with no redundancy?

From Pyramids to Haar

- Try it on the smallest possible case: two neighboring pixels, a and b.
- Compute both quantities at once:
- Two numbers in, two numbers out — no redundancy.
- The average m is a one-pixel “coarse version” of the pair (a two-pixel Gaussian level); the difference d is exactly what the average fails to capture (a two-pixel Laplacian level).
- And nothing is lost, because the pair is perfectly recoverable:

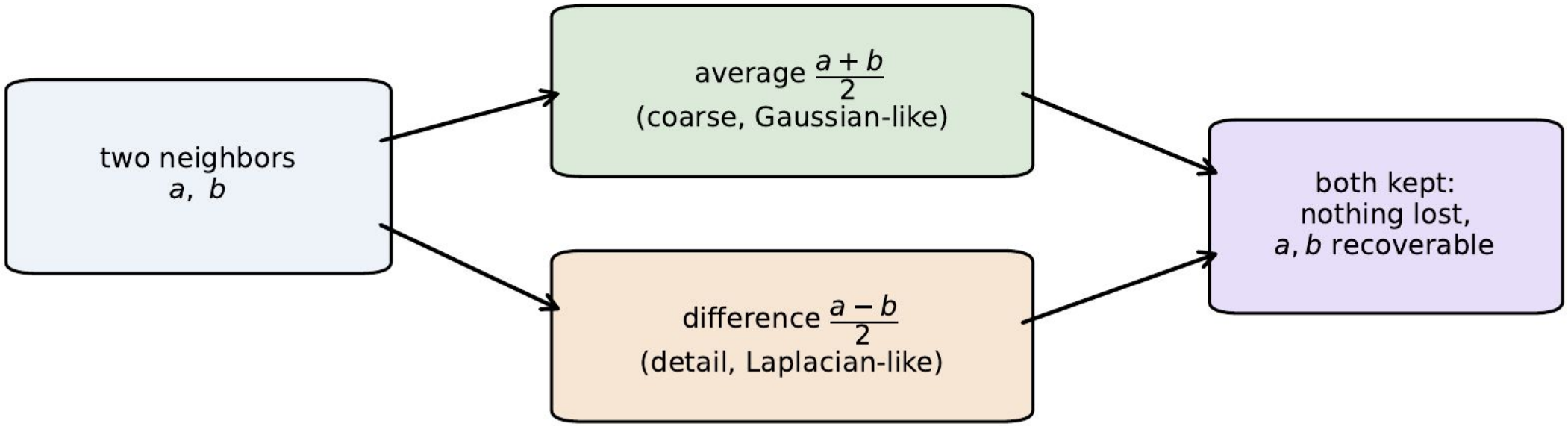
$$\text{average } m = \frac{a+b}{2}, \qquad \text{difference } d = \frac{a-b}{2}.$$

From Pyramids to Haar

- Perfect reconstruction, scale separation, and critically many numbers — the pyramid's virtues without its overhead, in two lines of arithmetic (the figure).

$$a = m + d, \qquad b = m - d.$$

From Pyramids to Haar



The average-and-difference block: from a pixel pair, compute the mean (coarse) and half-difference (detail) simultaneously. Two numbers in, two out, perfectly invertible — a Gaussian and Laplacian pyramid of a two-pixel image, with no redundancy.

From Pyramids to Haar

- This two-pixel block is the atom of the Haar transform: apply it to every pair, then recursively to the averages, and the pyramid ideas of this chapter reorganize into a genuine orthogonal transform — the very basis previewed in the R^2 example of §1.7 (whose “average and difference patterns” $1\sqrt{2}(1,1)$, $1\sqrt{2}(1,-1)$ are exactly this block, orthonormally scaled).
- The next section builds it in full.
- (Readers who know the signal-processing literature will recognize deeper machinery beneath this point; true to the chapter's rules, no filter-bank terminology will be used — averages and differences suffice.)

SECTION 9

The Haar Transform

One level in one dimension

- Let a signal (one image row, say) have values x_1, x_2, …, x_{2n}.
- The one-level Haar transform applies the average-and-difference block to each consecutive non-overlapping pair:
- The m_i are the average (approximation) coefficients — a half-length coarse version of the signal — and the d_i are the difference (detail) coefficients — what the coarse version misses.
- Total: 2n numbers in, n + n = 2n out, invertible pair by pair via $x_{2i-1} = m_i + d_i$, $x_{2i} = m_i - d_i$.

$$m_i = \frac{x_{2i-1} + x_{2i}}{2}, \qquad d_i = \frac{x_{2i-1} - x_{2i}}{2}, \qquad i = 1, \ldots, n.$$

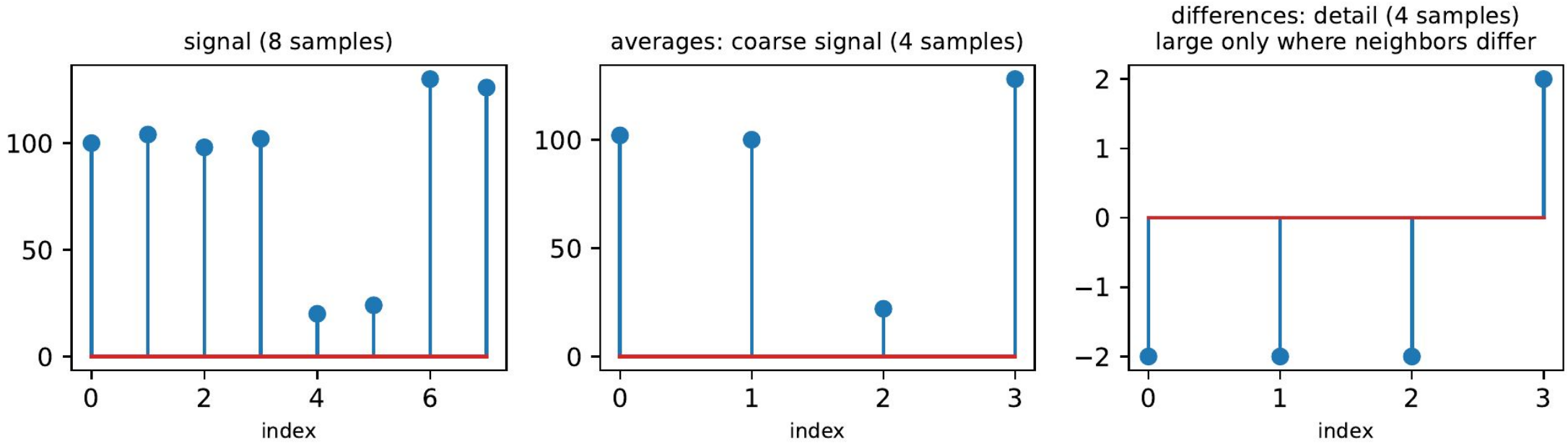
WORKED EXAMPLE

The complete numerical example

- Take the 8-sample signal Level 1.
- Pair up and compute averages and differences: Note what happened: the signal's shape (two high plateaus, one deep dip) lives in m; the four detail coefficients are tiny, because within each pair the neighbors nearly agree — neighboring samples of natural signals usually do (§1.7, once more).
- Level 2.
- Recurse on the averages m = (102, 100, 22, 128): Here a detail coefficient is finally large: $d'_2 = -53$ records the huge jump between the dip (22) and the last plateau (128) — a genuine edge at this scale, and the transform flags it.
- Level 3. Recurse once more on (101, 75): The full three-level Haar transform of x is the collection — eight numbers in, eight out, and six of the eight are small.

$$\mathbf{x} = (100,\ 104,\ 98,\ 102,\ 20,\ 24,\ 130,\ 126).$$

One level in one dimension



One Haar level on the example signal: the averages (center) form a half-length coarse signal preserving the shape; the differences (right) are near zero except where paired neighbors disagree.

Recursive decomposition

- The example already exercised the key structural idea: recursion.
- One Haar level splits a signal into (coarse half, detail half); the detail half is finished — it is pure scale-specific content, exactly like a Laplacian layer — but the coarse half is just a smaller signal, and the transform applies to it again.
- Recursing until the coarse part is a single number (the overall mean) yields the multi-level Haar decomposition: one mean, plus detail coefficients at every scale from coarsest to finest — precisely a Laplacian pyramid's worth of scale-separated information, in exactly 2n numbers with zero redundancy.
- The pyramid's construction (average down, subtract to get detail) and the Haar recursion (average pairs, difference pairs, recurse on averages) are the same idea; Haar is the pyramid made exact and economical.

Two dimensions

subband	content
average–average	coarse half-resolution image (“the average image”)
difference–average	horizontal-edge detail
average–difference	vertical-edge detail
difference–difference	diagonal / corner detail

Two dimensions

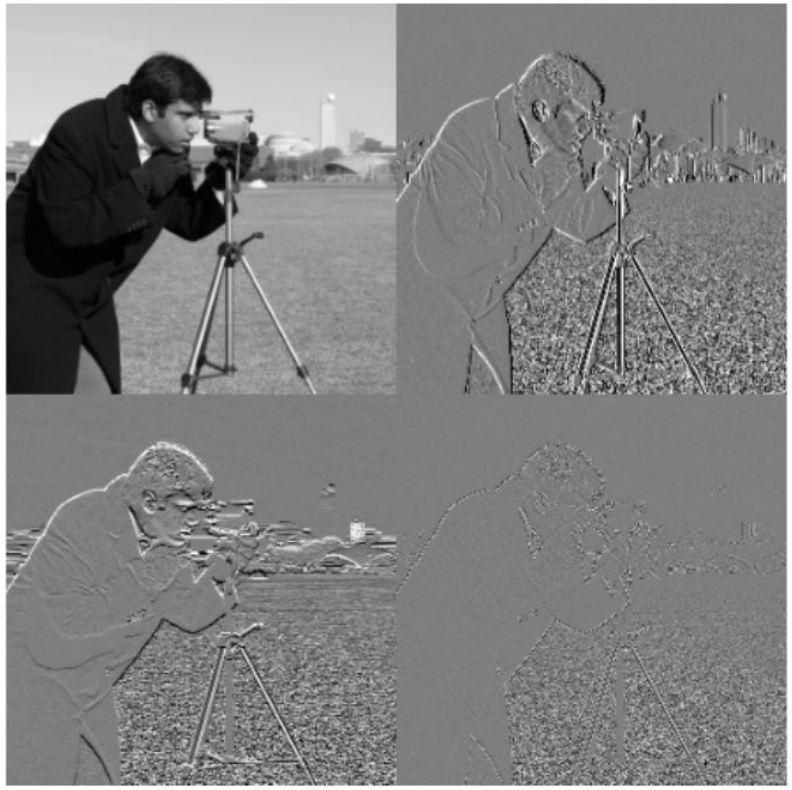
- Images are two-dimensional, and the extension is mechanical: apply the one-dimensional transform along every row, then along every column of the result.
- Each pass splits into averages and differences, so the two passes together split the image into four quarter-size subbands:
- (the figure; from here on the coefficients carry the orthonormal scaling 1√2(a± b) of §1.7 rather than 12(a± b), which changes nothing conceptually but makes the transform energy-preserving in the exact sense of Parseval, §1.6 — a fact §4.10 will lean on.) The recursion then acts on the average–average subband alone: transform it in place into four sub-subbands, and again, producing the nested layout of the figure.
- The result is the Haar coefficient image: same pixel count as the original, organized as one tiny average image plus directional detail at every scale.

Two dimensions

original 512^2

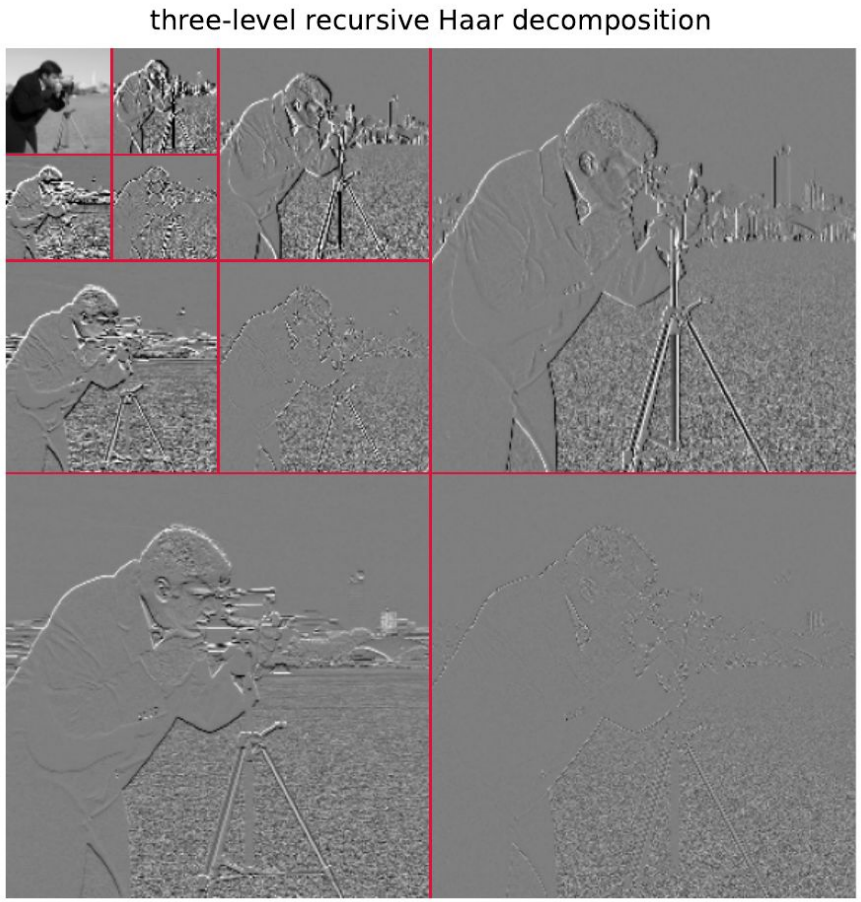


one Haar level: average | H-detail
V-detail | D-detail (each 256^2)



One two-dimensional Haar level. The image splits into four quarter-size subbands: a coarse average image (top left) and three detail subbands responding to horizontal, vertical, and diagonal structure (displayed with mid-gray = zero).

Two dimensions



Three-level recursive Haar decomposition (red lines mark the nesting). The average subband is repeatedly re-transformed; detail subbands at three scales surround the small coarse image in the top-left corner.

Two dimensions

- Look at the figure the way §4.6 looked at the Laplacian pyramid: the detail subbands are overwhelmingly mid-gray.
- Natural images are locally smooth almost everywhere, so almost every Haar difference — at every scale, in every direction — is nearly zero.
- The entire image has collapsed onto a small bright skeleton of significant coefficients: the coarse average plus the edges, at each scale, in each orientation.
- The transform of §1.7's promise — “a fixed basis in which a natural image has few large coefficients” — is now on the page, and the obvious question is what those few coefficients buy.

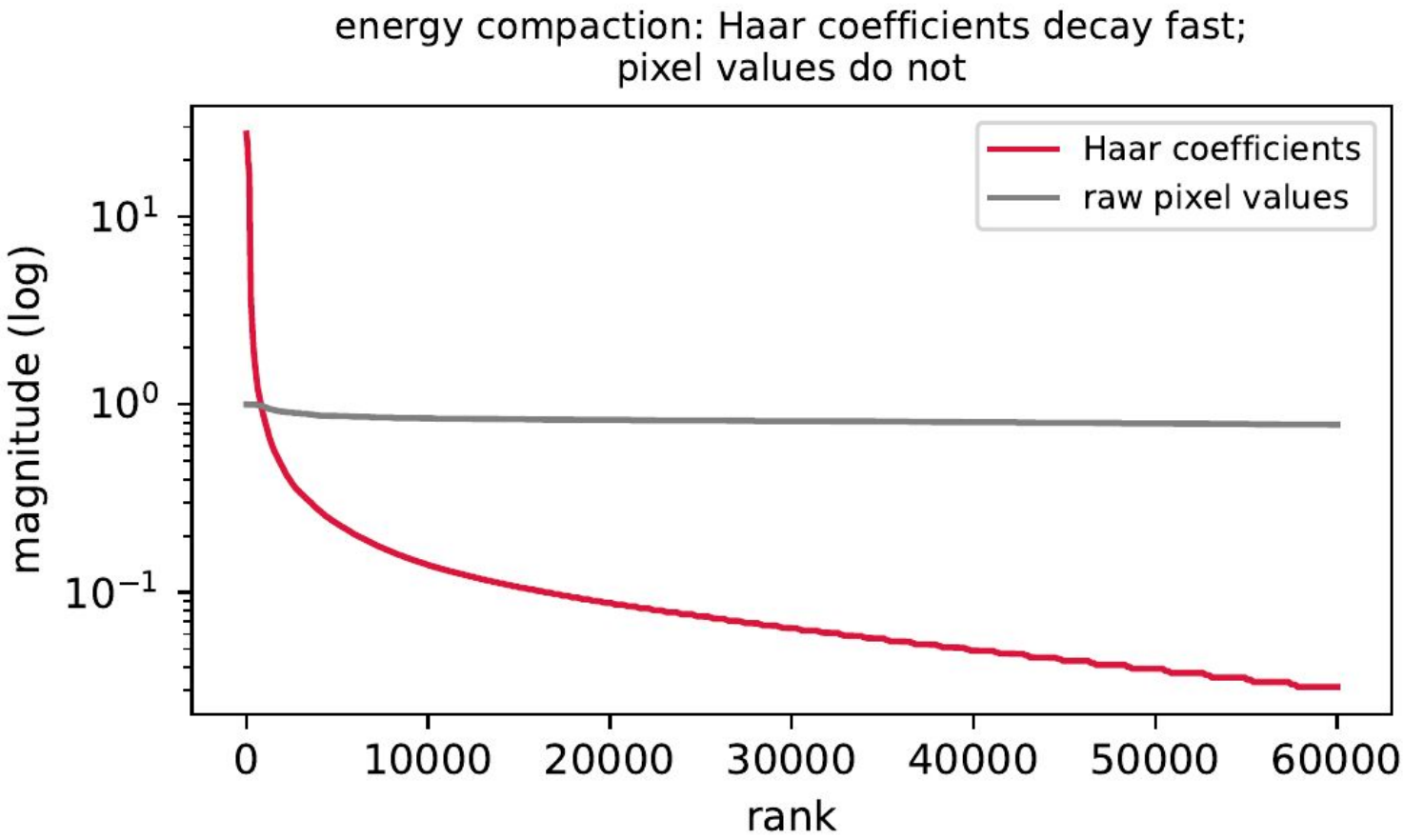
SECTION 10

Sparsity and Image Compression

Threshold and discard

- Make the observation quantitative.
- Compute a five-level Haar transform of the cameraman image and sort all 262,144 coefficient magnitudes: the decay is precipitous — a small population of large coefficients (the skeleton) and a vast population of near-zero ones (the smooth interior of every region, at every scale). the figure plots the sorted magnitudes against those of the raw pixels: pixel values, all of comparable size, decay hardly at all; Haar coefficients plunge by orders of magnitude.
- The transform has not changed the image's energy — by Parseval (§1.6), orthonormal Haar preserves it exactly — it has relocated it into few coefficients.
- The property is called energy compaction, and it is the entire mechanism of transform compression.

Threshold and discard



Energy compaction. Sorted magnitudes of the Haar coefficients versus the raw pixel values of the same image. Pixels are uniformly significant; Haar coefficients are dominated by a small fraction, with the rest near zero.

Threshold and discard

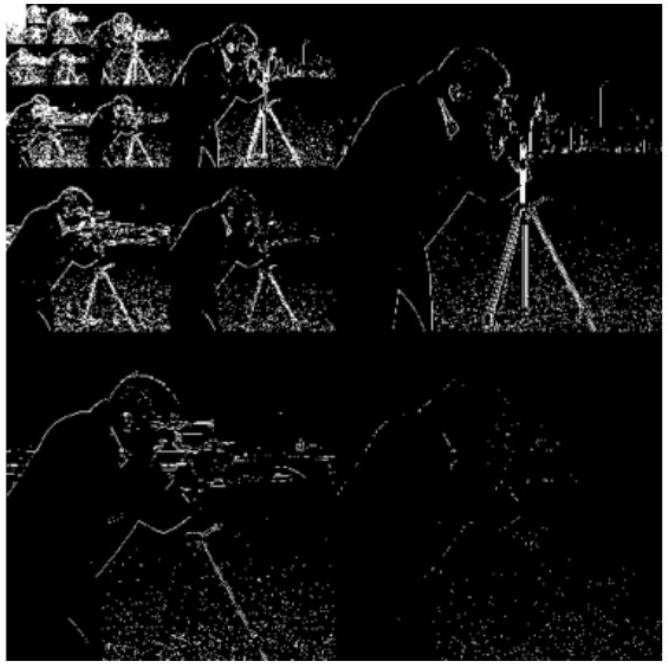
- The compression recipe writes itself: keep the large coefficients, discard the rest.
- Fix a threshold T and set every coefficient with $|c| < T$ to zero — coefficient thresholding — leaving a sparse representation: a short list of (position, value) pairs for the survivors.
- Reconstruction is the inverse Haar transform of the thresholded coefficients.
- The cost of the discard is known in advance and exactly, by Parseval: the squared reconstruction error equals the summed squares of the zeroed coefficients (§1.6's guarantee, now doing real work) — and because the zeroed coefficients were precisely the smallest ones, that error is as small as any equally-sparse choice could achieve.

Threshold and discard

coefficient magnitudes (log scale):
few bright, most near zero



locations of the kept 5%
of coefficients



reconstruction from 5% of
coefficients: rel. error 4.9%



Compression by thresholding. Left: coefficient magnitudes of the five-level Haar transform (log display) — the bright skeleton against a dark near-zero background. Center: the locations of the largest 5% of coefficients. Right: the image reconstructed from that 5% alone — relative error 4.9%, visually faithful.

Threshold and discard

- the figure executes the recipe: keeping only the largest 5% of coefficients — a 20× reduction in stored values — reconstructs the cameraman with 4.9% relative error, visually near-indistinguishable at normal size.
- Compare Chapter 1's rank-50 SVD compression (5× reduction, §1.9): the fixed Haar basis, costing no per-image computation of a decomposition, compresses this image markedly harder.
- The chapter's long construction has paid off in the book's first practical compression algorithm.

SECTION 11

Beyond Haar

Beyond Haar

- Haar is the simplest member of a large family — the wavelets — and honesty requires stating its rank in that family: the most instructive, and the crudest.
- Haar's building blocks are piecewise-constant: its average is a flat block, its difference an abrupt step (the figure, left).
- Consequently Haar represents flat regions with brutal efficiency but must spend many coefficients approximating gradual ramps and smooth curves — staircasing them — and heavily-compressed Haar reconstructions betray it with faint blockiness (visible, on close inspection, in the figure).
- Later wavelets fix exactly this, replacing the step-shaped building blocks with smoother ones while keeping everything structural — recursion across scales, perfect reconstruction, energy compaction, fast algorithms.
- The names to recognize:

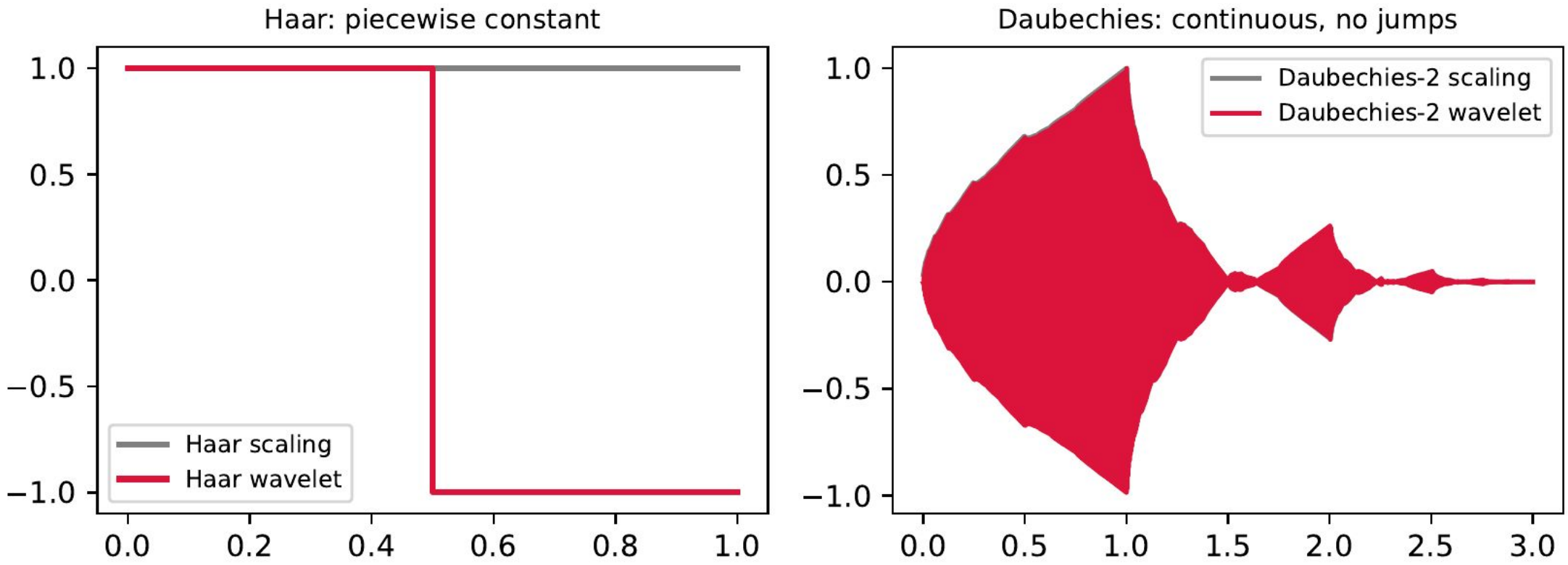
Beyond Haar

- Daubechies wavelets — the celebrated family of smooth, compactly-supported wavelets (the figure, right, shows the shortest member beyond Haar: continuous, no jumps);
- Symlets — Daubechies variants adjusted toward symmetry;
- Coiflets — variants whose averages behave especially well on smooth signals;
- Biorthogonal wavelets — pairs of complementary bases, exactly symmetric, and the family used in practical image coders.

Beyond Haar

- Smoother blocks represent smooth image content with far fewer significant coefficients, so these wavelets compress natural images visibly better than Haar and degrade more gracefully.
- We state this and stop — no derivations, no construction machinery, no filter banks; for every conceptual purpose in this book, a wavelet transform is “the Haar transform with better-shaped averages and differences,” and Haar remains our working example throughout.

Beyond Haar



Basis functions. Haar's (left) are piecewise constant — flat blocks and abrupt steps. The Daubechies-2 functions (right) are continuous and smooth-edged, so they represent gradual image variations with far fewer significant coefficients.

SECTION 12

Image Compression

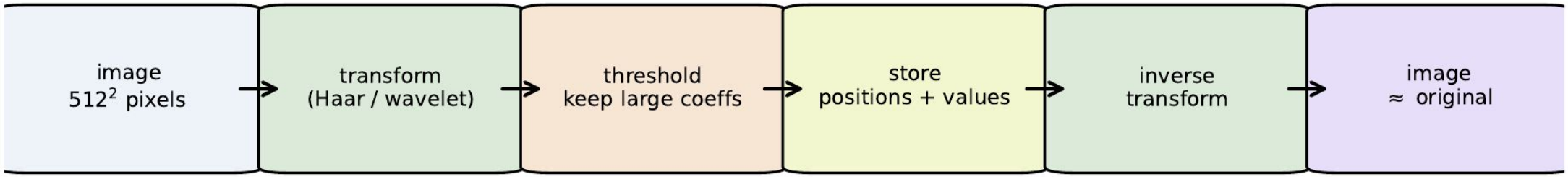
Image Compression

- The pieces assembled across this chapter snap together into the complete transform-compression pipeline (the figure):

Image Compression

- Transform. Apply a multi-level wavelet transform (Haar, or better) to the image — an invertible, energy-preserving change of representation into scale-and-orientation coefficients.
- Threshold. Zero the small coefficients; keep the sparse skeleton. This is the step that loses information — deliberately, measurably (Parseval prices it), and where it matters least.
- Store. Record the surviving coefficients' positions and values. (Real coders squeeze this step further — coarsely quantizing the kept values and entropy-coding the result — refinements of bookkeeping, not of concept.)
- Reconstruct. Inverse-transform the stored coefficients to display the image.

Image Compression



The transform compression pipeline: transform, threshold, store, inverse-transform. Every stage was built in this chapter.

Image Compression

- This is not a toy: it is, conceptually, JPEG2000, the wavelet-based image compression standard.
- JPEG2000 applies a multi-level biorthogonal wavelet transform (§4.11's last family), quantizes and entropy-codes the coefficients scale by scale, and inherits from the multiresolution structure exactly the virtues this chapter developed: strong compression from energy compaction, graceful degradation (dropping small coefficients blurs gently rather than breaking into blocks), and progressive transmission for free — send coarse-scale coefficients first, refine with finer scales, precisely the pyramid walk of the figure.
- The distance from this chapter's ideas to a deployed international standard is engineering, not concept.

SECTION 13

Looking Ahead

LOOKING AHEAD

Looking Ahead

- End on the number the chapter earned: a faithful cameraman from 5% of its Haar coefficients.
- Compression works because, in the right representation, most transform coefficients are unnecessary — the image's quarter-million pixels carry only a few thousand numbers' worth of essential content, and we now possess fixed, fast transforms that expose this.
- But sit with that fact for one more moment, because it is strange.
- The camera dutifully measured all 262,144 pixels.
- The compressor then immediately discarded 95% of what the transform revealed to be redundant.

Looking Ahead

- We acquire everything in order to throw almost everything away.

If only a few coefficients matter, why measure every pixel in the first place?

LOOKING AHEAD

Looking Ahead

- Could one instead acquire, from the outset, only about as many measurements as the image's true information content — and recover the image from those?
- The question sounds underdetermined to the point of hopelessness: fewer measurements than unknowns is precisely the ill-posed regime of §2.10.
- But the sparsity uncovered in this chapter is exactly the prior knowledge that regime demands, and the ℓ_1 machinery of §2.7 and §2.10 is exactly the tool that exploits it.
- The affirmative answer is called compressed sensing, and it is the next chapter.

EXERCISES

Conceptual

1. Why are multiple resolutions useful? Give three distinct motivations from §4.1 and explain, for each, what goes wrong with a single fixed resolution.

2. Why must images be smoothed before downsampling? Describe, in terms of the ring-pattern figure, what direct decimation does to structure finer than the new pixel spacing, and what smoothing changes.

3. Explain the difference between the Gaussian and Laplacian pyramids: what each level of each contains, how the two are related by construction, and which of the two is (nearly) sparse and why.

4. Explain the Haar transform to a reader who knows only Chapter 3: the two-pixel block, the role of averages versus differences, and the recursion. Where does the Laplacian pyramid's "detail layer" idea reappear in it?

5. Why does the Haar transform compress images? Your answer should use the terms energy compaction, thresholding, and Parseval, and explain why natural images in particular compress well.

6. Why are later wavelets (Daubechies and relatives) generally better than Haar? Identify the specific weakness of piecewise-constant building blocks and the specific image content on which it costs coefficients.

EXERCISES

Numerical

1. Construct a Gaussian pyramid, by hand, for the 4-pixel signal (80, 84, 20, 24), using plain pair-averaging as reduce: give G_0, G_1, G_2.

2. For the same signal, compute one level of a Laplacian pyramid: using nearest-neighbor as expand (repeat each coarse value twice), compute $L_0 = G_0$ - expand(G_1), and verify that L_0 + expand(G_1) recovers G_0 exactly.

3. Perform a complete Haar transform, by hand, of the signal (50, 54, 90, 86, 12, 16, 60, 56): give the coefficients at every level, in the layout of the worked example of §4.9. Which coefficients are large, and what feature of the signal does each large one record?

4. Reconstruct the original signal from the Haar coefficients (40; 10; 4, -6; 1, -1, 2, 0) (mean; coarsest detail; mid details; finest details), showing each level of the inversion.

5. In the previous exercise, set the four finest detail coefficients to zero and reconstruct again. Compute the squared reconstruction error directly (sum of squared differences from the true signal) and verify that it equals the sum of squares of the discarded coefficients, as Parseval predicts (assume orthonormal scaling, i.e., take the stated coefficients as exact).

EXERCISES

Programming

1. Build a five-level Gaussian pyramid of a photograph (Gaussian smoothing with $\sigma = 1$, then decimation by 2) and display the levels at true relative sizes.

2. Construct the corresponding Laplacian pyramid using bilinear expand, displaying the detail layers with mid-gray as zero.

3. Reconstruct the image from its Laplacian pyramid and report the relative Frobenius error (§1.5); it should be at machine precision.

4. Implement the two-dimensional Haar transform (rows, then columns; orthonormal scaling) and its inverse, and verify perfect reconstruction on a test image.

5. Compress an image by five-level Haar thresholding: keep the largest p% of coefficients for $p \in \{1, 5, 10, 25\}$, reconstruct each, and display the results with their relative errors.

6. For the same experiment, plot relative error against fraction kept (log–log), and separately verify numerically, for one threshold, that the squared error equals the sum of squares of the discarded coefficients.

KEY TAKEAWAYS

Multiresolution Image Representations

- From Pyramids to Haar
- The Haar Transform
- Sparsity and Image Compression
- Beyond Haar
- Image Compression
- Looking Ahead

End of Part